PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Registro de asistencia estudiantil

Integrantes:

Rodrigo Escudero

Mateo Cueto

Gonzalo Leal

Diego Binimelis

18 de abril de 2026

Índice

Lista de figuras ..... **II**

Lista de tablas ..... **III**

1 ¿Qué datos maneja el sistema? (SIA-1) **2**

2 Las clases que construyen el sistema (SIA-2) **3**

3 Buenas prácticas (SIA-3) **5**

4 Uso de colecciones de Java (SIA-4) **6**

5 Sobrecarga de métodos (SIA-5) **8**

6 Sobreescritura y polimorfismo (SIA-6) **9**

7 Funcionalidades básicas del menú (SIA-7) **10**
- 7.1 Modo Consola 10
- 7.2 Modo Gráfico (Swing) 10

8 Editar, eliminar y buscar (SIA-8) **12**

9 Funcionalidad propia del programa (SIA-9) **13**

10 Consola o ventanas, a eleccion (SIA-10) **16**

11 Los datos no se pierden al cerrar (SIA-11) **19**

12 Manejando errores sin que el programa se caiga (SIA-12) **20**

13 Control de versiones con GitHub (SIA-13) **22**

Índice de figuras

Figura 2.1 Diagrama de Clases del sistema 4
Figura 4.1 Diagrama de Estructuras Principales del sistema 6
Figura 4.2 Diseño del TreeMap listaGlobal de estudiantes 6
Figura 4.3 Diseño del ArrayList listaAsistencia 7
Figura 7.1 Vista previa del menu en modo ventana 11
Figura 9.1 Diagrama de flujo del proceso de ordenamiento y visualización de la lista de curso 15
Figura 10.1 Selector de modo de arranque 16
Figura 10.2 Vista del menu en modo consola 18

Índice de tablas

Tabla 9.1 Funcionalidades propias del negocio 13
Tabla 10.1 Diferencias y similitudes entre los modos de ejecución 17
Tabla 12.1 Excepciones personalizadas del sistema 20

Introducción

Este documento describe el desarrollo del **Sistema de Información de Asistencia Estudiantil**, una aplicación en Java que permite gestionar el registro de asistencia de alumnos en un colegio. El sistema maneja estudiantes, cursos y distintos tipos de asistencias (normal, inasistencia justificada, salida anticipada). Ofrece dos modos de interacción: consola (CLI) y ventanas (Swing), y guarda los datos en un archivo CSV para su persistencia. A continuación se detalla cómo cada componente del código cumple con los requisitos académicos SIA.

1. ¿Qué datos maneja el sistema? (SIA-1)

Para administrar la asistencia, necesitamos representar tres cosas principales:

- **Estudiante**: tiene un RUT (que lo identifica de forma única), nombre completo, edad y el curso al que pertenece.
- **Asistencia**: es un registro de un día específico. Guarda la fecha, una observación y, según el tipo, datos extra (por ejemplo, si fue puntual o el motivo de la falta).
- **Curso**: agrupa a los estudiantes del mismo nivel (por ejemplo, "1º Básico").

La relación entre ellos es sencilla: un **Curso** contiene varios **Estudiantes**, y cada **Estudiante** guarda una lista con todas sus **Asistencias** a lo largo del año.

2. Las clases que construyen el sistema (SIA-2)

A continuación se muestran las clases Java que modelan los datos anteriores. Cada una tiene atributos (variables que guardan información) y métodos (funciones que realizan acciones).

Clase Persona (datos básicos de cualquier persona)

¿Qué hace? Guarda los datos personales que comparten tanto estudiantes como (en un futuro) profesores. Además, incluye un método para comprobar si un RUT es válido (usando el algoritmo del dígito verificador).

Clase Estudiante (un alumno concreto)

¿Qué hace? Extiende a `Persona` añadiendo el curso del alumno y una lista donde se irán guardando todos sus registros de asistencia. Proporciona métodos para agregar un nuevo registro, buscar uno existente o contar cuántos hay de cada tipo.

Jerarquía de Asistencia (distintos tipos de registro)

¿Qué hace? `Asistencia` es una clase abstracta (no se puede crear directamente) que define lo común a todos los registros: ID, fecha y observación. Luego tenemos tres variantes:

- `AsistenciaNormal`: añade si fue puntual.
- `InasistenciaExtraordinaria`: añade el motivo de la falta.
- `SalidaAnticipada`: añade la hora de salida.

Cada una sobreescribe `getResumen()` para devolver un texto personalizado.

Clase Curso (conjunto de estudiantes)

¿Qué hace? Representa un curso (ej. "3º Medio"). Internamente usa un `TreeMap` que asocia el RUT con el objeto `Estudiante`, manteniéndolos ordenados por RUT automáticamente.

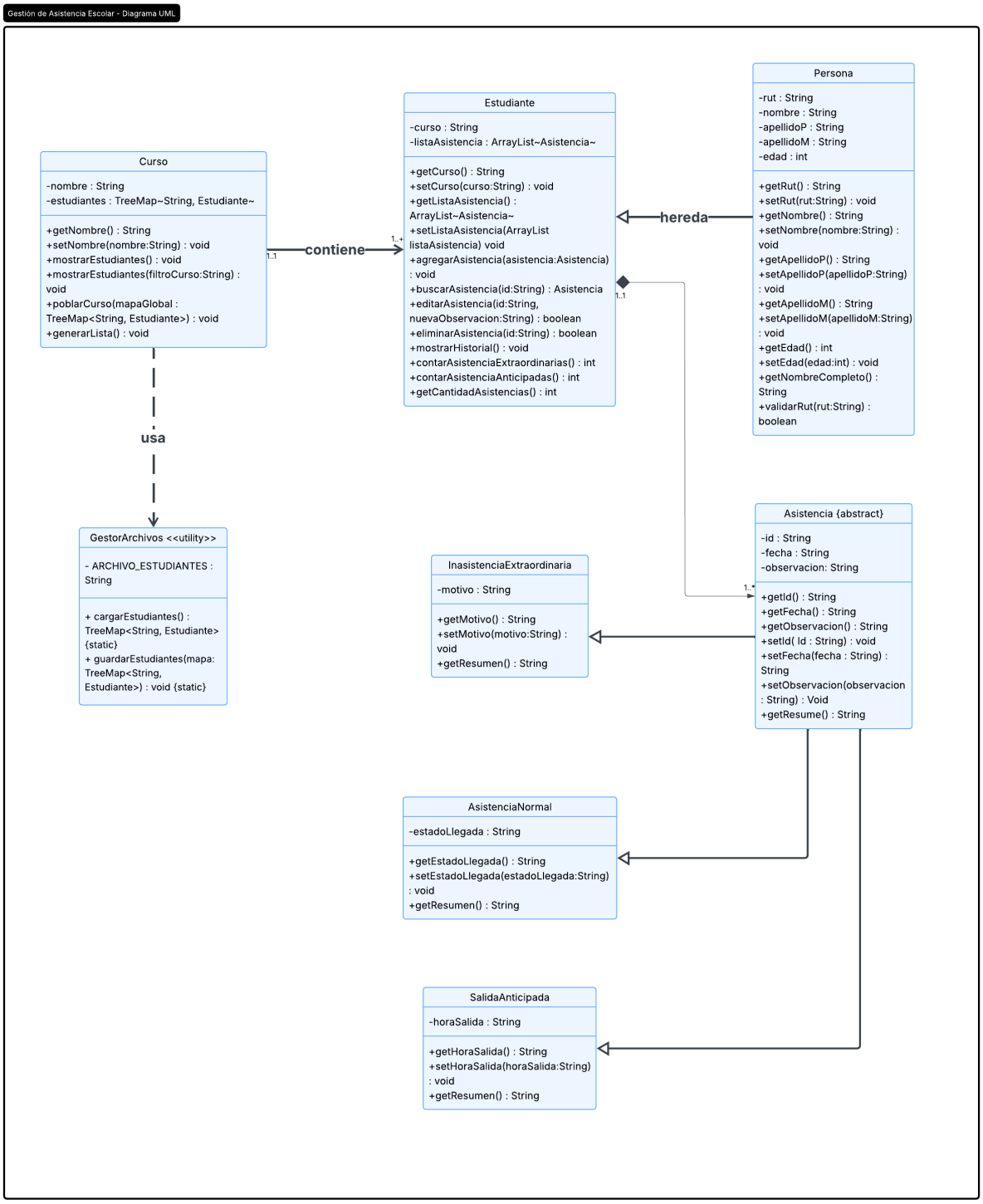

Figura 2.1: Diagrama de Clases del sistema

3. Buenas prácticas (SIA-3)

- **Atributos privados**: todas las clases del modelo usan `private` con getters/setters públicos.
- **Código modular**: separación clara entre modelo (Persona, Estudiante, Asistencia...), persistencia (GestorArchivos) e interfaz (Ventana, Paneles).
- **Datos iniciales**: al iniciar, `GestorArchivos.cargarEstudiantes()` lee el archivo `estudiantes.csv` y puebla el mapa global.

4. Uso de colecciones de Java (SIA-4)

El sistema guarda la información en dos niveles de colecciones:

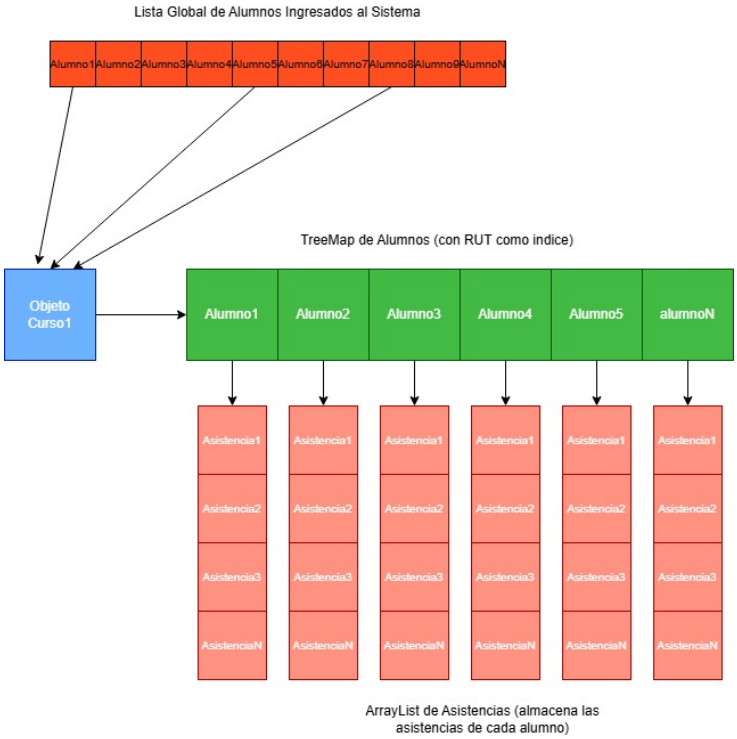

Figura 4.1: Diagrama de Estructuras Principales del sistema

1. **Primer nivel – Mapa global de estudiantes:**

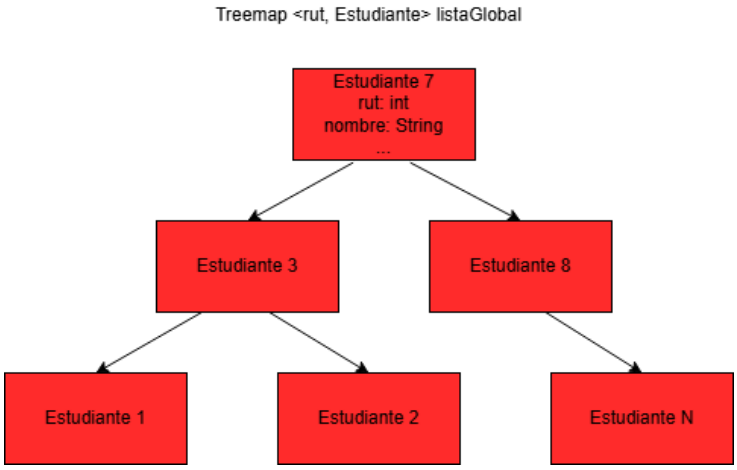

Figura 4.2: Diseño del TreeMap listaGlobal de estudiantes

Es un `TreeMap` donde la **clave** es el RUT (String) y el **valor** es el objeto `Estudiante`. Al usar `TreeMap`, los estudiantes quedan ordenados por RUT automáticamente y podemos encontrar a cualquiera rápidamente.

2. **Segundo nivel – Lista de asistencias dentro de cada estudiante:**

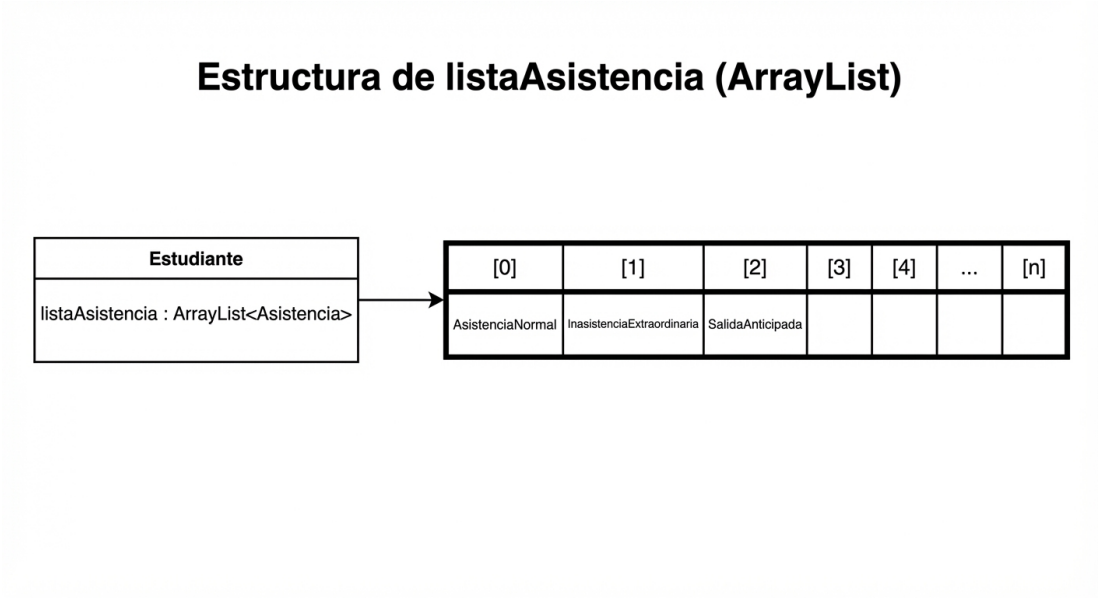

Figura 4.3: Diseño del ArrayList listaAsistencia

Cada `Estudiante` tiene un `ArrayList` que guarda sus registros de asistencia en el orden en que se fueron agregando (cronológico).

Anidamiento: El mapa global contiene estudiantes, y cada estudiante contiene su lista de asistencias. Es una estructura de çolección dentro de otra colección".

5. Sobrecarga de métodos (SIA-5)

La sobrecarga consiste en tener varios métodos con el mismo nombre pero distintos parámetros. El compilador decide cuál ejecutar según lo que le pasemos. Aquí hay dos ejemplos claros:

En la clase Curso

La clase `mostrarEstudiantes()` nos muestra los estudiantes ingresados hasta el momento en el sistema.

Si llamamos a `mostrarEstudiantes()` sin argumentos, vemos todos los alumnos de todos los cursos, en cambio, si le pasamos un nombre de curso, filtra por ese nombre.

En la clase Estudiante

El metodo agregarAsistencia, este metodo genera un objeto en base al tipo de asistencia que se requiera (normal, extraordinaria o retiro).

La primera versión es flexible (podemos pasar cualquier tipo de asistencia). La segunda es un atajo para el caso más común: registrar que el alumno asistió normalmente.

6. Sobreescritura y polimorfismo (SIA-6)

Cuando una clase hija redefine un método de la clase padre, hablamos de **sobreescritura**. En nuestro sistema, la clase `Asistencia` define un método `getResumen()`, y cada subclase lo cambia para dar su propio mensaje:

- `AsistenciaNormal`: "Vino a clases y llegó temprano.º "...pero no llegó temprano".
- `InasistenciaExtraordinaria`: "Faltó. Motivo: enfermedad".
- `SalidaAnticipada`: "Salió a las 14:30".

Polimorfismo en acción: cuando recorremos la lista de asistencias y hacemos `a.getResumen()`, Java ejecuta automáticamente la versión correspondiente al tipo real del objeto `a`. Así, con una sola línea de código obtenemos la descripción adecuada.

7. Funcionalidades básicas del menú (SIA-7)

El sistema ofrece, tanto en consola como en interfaz gráfica, las operaciones de **inserción manual** y **mostrar listado** para ambas colecciones.

7.1. Modo Consola

El menú principal (clase `Main`) incluye las siguientes opciones:

- **Opción 1 – Agregar Estudiante**: solicita los datos del alumno, valida el RUT y la edad, y lo inserta en `Main.listaGlobal`.
- **Opción 2 – Listar Estudiantes**: recorre el `TreeMap` global y muestra todos los alumnos registrados.
- **Opción 6 – Registrar Asistencia**: pide el RUT del estudiante, fecha, observación y tipo de registro; crea un objeto de la subclase correspondiente y lo agrega a la lista de asistencias del alumno.
- **Opción 7 – Ver Historial de Asistencias**: muestra todas las asistencias de un estudiante (usando el método polimórfico `getResumen()`).

7.2. Modo Gráfico (Swing)

- **Panel `IngresoEstudiante`**: formulario para agregar un nuevo estudiante. Al presionar *Agregar Estudiante*, se crea una instancia de `Estudiante` y se almacena en `Main.listaGlobal`.
- **Panel `PanelGestionEstudiantes`**: contiene una tabla (`JTable`) que muestra todos los estudiantes cargados desde el mapa global. El método `cargarTabla()` itera sobre `Main.listaGlobal.values()` y añade filas al modelo.
- **Panel `PanelAsistencias`**: al cargar el historial de un alumno (ingresando su RUT), se llena una tabla con los registros de asistencia, usando `getResumen()` para la columna de resumen. El botón *Registrar Asistencia* abre diálogos para ingresar los datos y añade el nuevo objeto a la lista del estudiante.

Ambos entornos cumplen con el requisito de permitir la inserción manual y la visualización de los elementos de ambas colecciones.

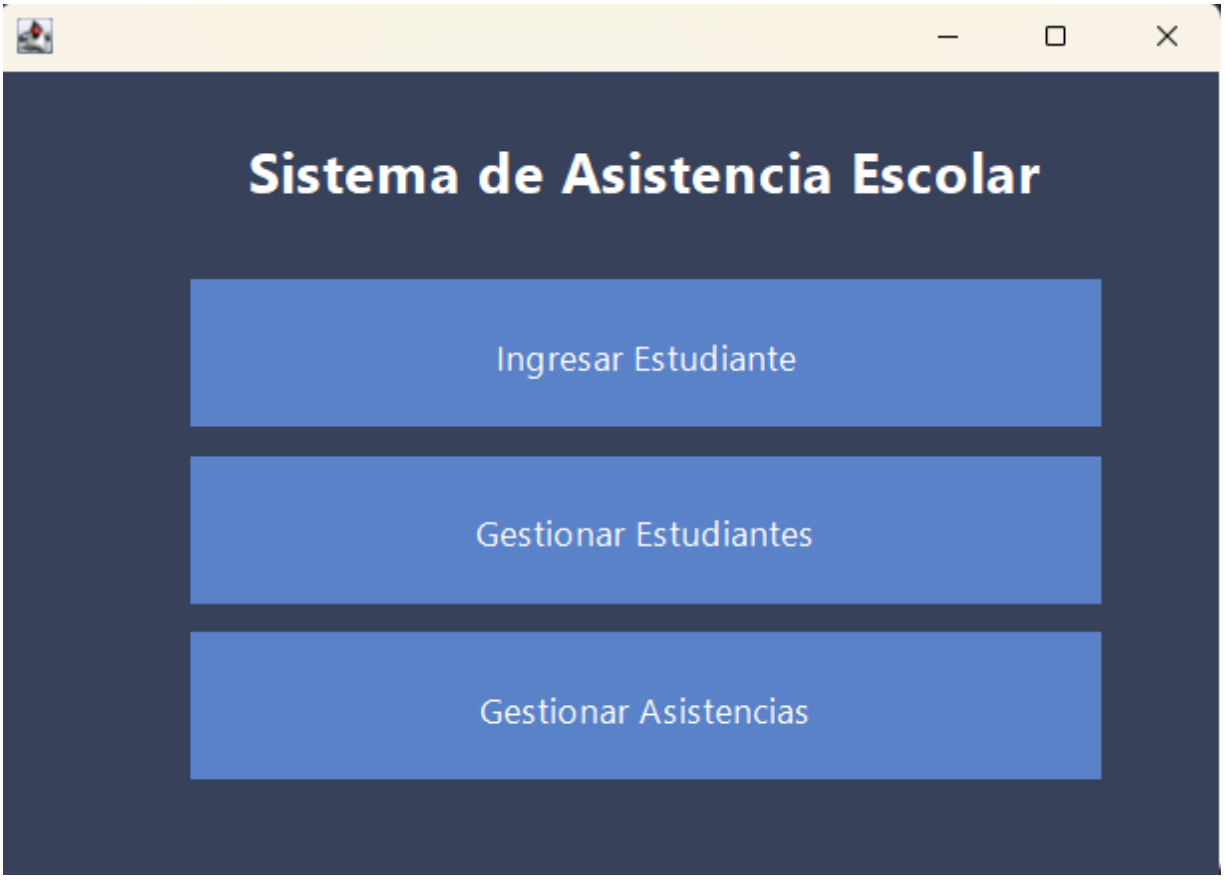

Figura 7.1: Vista previa del menu en modo ventana

8. Editar, eliminar y buscar (SIA-8)

Además de agregar y listar, el sistema permite modificar y eliminar información, así como buscar elementos específicos.

Edición

- **Estudiante**: modificar nombre/apellidos (consola opc. 4, GUI botón Editar en tabla).
- **Asistencia**: cambiar la observación de un registro (consola opc. 8, GUI botón Editar Observación en tabla de asistencias).

Eliminación

- **Estudiante**: borrar por RUT (consola opc. 5, GUI botón Eliminar en tabla de estudiantes).
- **Asistencia**: borrar por ID (consola opc. 9, GUI botón Eliminar en tabla de asistencias).

Búsqueda

- **Estudiante**: buscar por RUT y mostrar sus datos (consola opc. 3, GUI campo de texto + botón Buscar).
- **Asistencia**: buscar por ID dentro del historial de un alumno (consola opc. 10, en GUI se busca visualmente en la tabla).

Todas estas operaciones utilizan los métodos que vimos en las clases del modelo: `Main.obtenerEstudiantePorRut()`, `Estudiante.buscarAsistencia()`, etc.

9. Funcionalidad propia del programa (SIA-9)

Además de las operaciones básicas de alta, baja, modificación y consulta, el sistema incluye dos herramientas pensadas específicamente para facilitar la gestión escolar diaria. Ambas aprovechan los datos almacenados en las colecciones para ofrecer información valiosa sin necesidad de escribir código adicional.

Funcionalidad	**¿Qué hace?**	**¿Dónde se encuentra?**
Detección de exceso de inasistencias	Recorre todos los estudiantes, cuenta sus inasistencias justificadas y lista aquellos que superan un límite definido por el usuario.	Menú consola: opción 12
Lista ordenada de un curso	Genera una nómina de los estudiantes de un curso específico, ordenada alfabéticamente por apellido paterno, materno y nombre.	Menú consola: opción 11

Tabla 9.1: Funcionalidades propias del negocio

1. Alumnos con exceso de inasistencias

Esta herramienta ayuda a identificar tempranamente a los estudiantes en riesgo de repitencia por faltas reiteradas. El flujo de trabajo es el siguiente:

¿Cómo detecta el sistema a los alumnos con muchas inasistencias?

1. El usuario ingresa un número máximo de inasistencias permitidas (por ejemplo, 5).
2. El sistema recorre uno por uno a todos los estudiantes registrados en el `TreeMap` global.
3. Para cada estudiante, se invoca un método interno que cuenta cuántos registros de tipo *InasistenciaExtraordinaria* tiene en su historial.
4. Si la cantidad de inasistencias del alumno supera el límite indicado, se muestra en pantalla su nombre completo junto con el número exacto de faltas.
5. Si ningún alumno supera el límite, se informa que no hay casos críticos.

Este proceso no requiere modificar la base de datos ni escribir consultas complejas: simplemente aprovecha la lista de asistencias que ya posee cada estudiante.

2. Generación de lista ordenada por curso

Esta funcionalidad permite obtener rápidamente la nómina de un curso en orden alfabético, ideal para pasar asistencia en clase o elaborar informes oficiales. El procedimiento es igualmente sencillo:

1. El usuario selecciona un curso de la lista predefinida (desde 1º Básico hasta 4º Medio).
2. El sistema filtra a todos los estudiantes cuyo atributo `curso` coincida con el elegido y los coloca en una colección temporal.
3. A continuación, se ordena esa colección utilizando un criterio en cascada:
   - Primero por apellido paterno.
   - Luego por apellido materno.
   - Finalmente por nombre.
4. La lista ordenada se muestra numerada en consola.

El siguiente diagrama de flujo ilustra visualmente los pasos que sigue el método encargado de generar la lista:

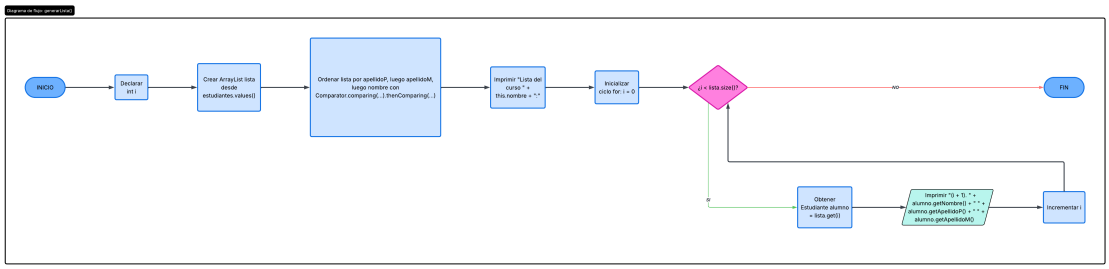

Figura 9.1: Diagrama de flujo del proceso de ordenamiento y visualización de la lista de curso

En resumen: ambas funcionalidades demuestran que la estructura de datos anidada del sistema permite generar información útil para la gestión escolar a partir de los registros ya almacenados.

10. Consola o ventanas, a eleccion (SIA-10)

El sistema ofrece al usuario la posibilidad de elegir entre dos modos de ejecución al iniciarse: mediante una **interfaz de consola** (línea de comandos) o una **interfaz gráfica** (ventanas Swing). Ambos modos comparten exactamente la misma lógica.

Selector de modo al arrancar

Al ejecutar el programa, aparece un sencillo menú de selección:

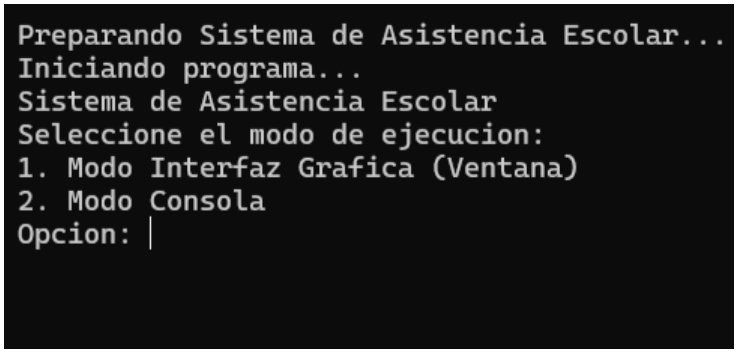

Figura 10.1: Selector de modo de arranque

Dependiendo del número ingresado, se inicia la versión correspondiente.

Comparativa de modos

Característica	Modo Consola	Modo Gráfico (Swing)
Entrada de datos	Mediante `Scanner` (teclado). El usuario escribe comandos numéricos y texto.	Mediante campos de texto, botones, tablas y cuadros de diálogo.
Salida de información	Texto plano en la terminal con `System.out.println`.	Ventanas, tablas (`JTable`) y mensajes emergentes (`JOptionPane`).
Navegación	Menú numérico anidado. Se selecciona una opción y se ejecuta la acción.	Paneles intercambiables con `CardLayout` y botones para avanzar/retroceder.
Flujo de trabajo	Lineal: el programa espera a que el usuario ingrese un dato antes de continuar.	Orientado a eventos: el usuario dispara acciones al hacer clic en botones o seleccionar filas.
Misma lógica	Usa las clases `Main.listaGlobal`, `Estudiante`, `Asistencia`, etc.	Usa exactamente las mismas clases del modelo.
Persistencia	Al elegir "Guardar y Salir" (opción 0) se llama a `guardarEstudiantes()`.	Al cerrar la ventana (WindowListener) se invoca automáticamente el mismo método de guardado.

Tabla 10.1: Diferencias y similitudes entre los modos de ejecución

Estructura del modo gráfico

La interfaz Swing se organiza en paneles contenidos en un `JFrame` principal (`Ventana`) que utiliza un `CardLayout` para cambiar entre las siguientes vistas:

- **Menú principal**: botones grandes para acceder a las secciones.
- **Ingreso de estudiantes**: formulario con campos de texto y un combobox de cursos.
- **Gestión de estudiantes**: tabla con todos los alumnos, botones para buscar, editar y eliminar.
- **Gestión de asistencias**: panel para cargar el historial de un alumno por RUT, tabla de registros y acciones (agregar, editar, eliminar).

La navegación se controla con el método `cambiarVista(nombre)` y se mantiene un historial para permitir volver a la pantalla anterior.

Esquema del flujo en modo consola

```
Menu Principal
Estudiantes:
 1. Agregar Estudiante
 2. Listar Estudiantes
 3. Buscar Estudiante por RUT
 4. Editar Estudiante
 5. Eliminar Estudiante
Asistencias:
 6. Registrar Asistencia
 7. Ver Historial de Asistencias
 8. Editar Observacion de Asistencia
 9. Eliminar Asistencia
10. Buscar Asistencia por ID
Reportes:
11. Generar Lista Ordenada de Curso
12. Alumnos con exceso de inasistencias
Sistema:
 0. Guardar y Salir
Opcion: |
```

Figura 10.2: Vista del menu en modo consola

Cada opción ejecuta un método estático de la clase `Main` que interactúa con el usuario a través de la consola.

Ventaja del diseño dual

- El **modo consola** es ideal para entornos sin interfaz gráfica, automatización de pruebas o usuarios avanzados que prefieren el teclado.
- El **modo gráfico** resulta más intuitivo para usuarios finales (profesores, administrativos) que esperan una experiencia visual con tablas y botones.
- Ambos modos operan sobre la misma colección de datos (`Main.listaGlobal`), por lo que se puede iniciar en un modo, hacer cambios, cerrar, y luego abrir en el otro modo sin pérdida de información.

11. Los datos no se pierden al cerrar (SIA-11)

Para que la información no desaparezca cuando apagamos el computador, el sistema guarda todo en un archivo `estudiantes.csv` dentro de la carpeta `resources/`.

- **Al iniciar**: la clase `GestorArchivos` lee el archivo y reconstruye todos los objetos `Estudiante`, colocándolos en `Main.listaGlobal`.
- **Al salir**: ya sea que cerremos la ventana o elijamos la opción "Guardar y Salir.[en] consola, se llama a `GestorArchivos.guardarEstudiantes()`, que escribe el contenido actual del mapa en el mismo archivo CSV.

El formato del archivo es simple: cada línea contiene `rut;nombre;apellidoP;apellidoM;edad;curso`. Esto permite que sea fácil de leer incluso con un editor de texto.

12. Manejando errores sin que el programa se caiga (SIA-12)

Para evitar que el programa falle ante situaciones como un RUT mal escrito o un estudiante inexistente, se definieron **dos excepciones personalizadas** y se utilizan bloques ‘try-catch‘ tanto en consola como en ventanas.

Excepciones propias del sistema

Excepción	**¿Cuándo se lanza?**
`EstudianteNoEncontradoException`	Al buscar un estudiante por RUT y no hallarlo en el mapa global.
`RutInvalidoException`	Al ingresar un RUT que no cumple con el formato o dígito verificador chileno.

Tabla 12.1: Excepciones personalizadas del sistema

Manejo en diferentes partes del programa

- **Consola (`Main.java`)**:
  - Al agregar estudiante: se valida RUT con `RutInvalidoException`. Si falla, se muestra mensaje y se pide reintentar.
  - Al buscar/editar/eliminar: se captura `EstudianteNoEncontradoException` y se imprime un aviso sin interrumpir el menú.
  - Al leer edad: se usa ‘try-catch‘ para ‘NumberFormatException‘ y evitar que texto no numérico rompa el flujo.
- **Interfaz gráfica (Swing)**:
  - En `PanelGestionEstudiantes` y `PanelAsistencias`, si no se encuentra un RUT, se muestra un diálogo con `JOptionPane` en lugar de imprimir en consola.
  - En `IngresoEstudiante`, la edad se valida igual que en consola.
- **Persistencia (`GestorArchivos`)**:
  - Al leer/escribir el CSV se capturan ‘IOException‘ y ‘NumberFormatException‘, imprimiendo un error en consola pero permitiendo que la aplicación siga funcionando (con datos vacíos si el archivo no existe).

Ejemplo representativo

En lugar de mostrar múltiples fragmentos de código, el siguiente esquema resume el flujo típico de manejo de errores al buscar un estudiante:

Búsqueda de estudiante por RUT
1. El usuario ingresa un RUT.
2. Se invoca `Main.obtenerEstudiantePorRut(rut)`.
3. Si el RUT no está en el `TreeMap`, se **lanza** `EstudianteNoEncontradoException`.
4. El bloque `catch` correspondiente **captura** la excepción y muestra un mensaje claro al usuario.
5. El programa continúa su ejecución normal (vuelve al menú o a la pantalla anterior).

13. Control de versiones con GitHub (SIA-13)

El proyecto se gestionó mediante un repositorio en GitHub, accesible en:

https://github.com/YungRodri/proyectoAsistenciaEscolar